Звіт: структура даних для багатопотокової програми (варіант 14, m = 2)

1. Виконавець: Гнидка Дем'ян Євгенійович, група К27.

2. Умова (варіант 14): структура з m = 2 цілих полів, кожне з яких можна читати та змінювати окремо, і перевантажений operator string для всієї структури. Частоти: поле 0 — read 25%, write 5%; поле 1 — read 5%, write 25%; string 40%. Атомарні змінні заборонені. Згенеровано файли для трьох послідовностей дій: (а) частоти відповідають умові; (б) всі частоти рівні; (в) частоти, далекі від умови (read 0 — 2%, write 0 — 45%, read 1 — 40%, write 1 — 3%, string — 10%). Виміряно час виконання в 1, 2 і 3 потоках; загальна кількість операцій у кожному випадку 6 000 000, час зчитування файлів не враховано.

3. Схема захисту даних. Використано два м'ютекси (std::mutex), по одному на кожне поле. Кожен м'ютекс разом зі значенням поля розміщено у власному блоці alignas(64), щоб поля не ділили один кеш-рядок (false sharing). Методи get і set беруть лише м'ютекс свого поля. operator string бере обидва м'ютекси в фіксованому порядку (поле 0, потім поле 1), що унеможливлює взаємоблокування; значення копіюються, м'ютекси звільняються, і лише після цього формується рядок. Обґрунтування: поля незалежні, тому операція над одним полем не чекає на інше, а в сценарії (а) операції read 0 і write 1 разом складають 50% і переважно не конфліктують. Один глобальний м'ютекс цього не забезпечує. Варіант з shared_mutex для поля 0 (читання домінує) не використано: у вимірюваннях він був повільнішим за звичайний м'ютекс, до 4 разів на двох потоках. Перевірку ThreadSanitizer пройдено без повідомлень про data race.

4. Результати (середній час, мс, 5 запусків):

Сценарій	**1 потік**	**2 потоки**	**3 потоки**
а) частоти з умови	150.6	778.0	396.5
б) усі частоти рівні	161.3	648.2	276.3
в) далекі від умови	130.4	518.0	260.9

5. Висновки. Таблиця не відповідає очікуванню прискорення: найшвидшим є один потік, а два потоки працюють у 3–5 разів повільніше, ніж один. Кожна операція дуже коротка, тому накладні витрати на захоплення м'ютексів і перемикання потоків переважають користь від паралельного виконання. Сценарії з великою часткою string (40% в сценарії а) гірші, бо string блокує обидва поля. Три потоки швидші за два, але повільніші за один. Для прискорення потрібна структура з меншим конфліктом або довші операції між блокуваннями.

6. Виконано самостійно: спроєктовано структуру з двома м'ютексами та вирівнюванням кеш-рядків; написано генератор трьох сценаріїв з вагами, заданими умовою; написано бенчмарк, що виконує файли в 1, 2 і 3 потоках (кожен потік працює зі своїми даними, загальна кількість операцій однакова) і виводить час для кожного файлу; сценарій (в) підібрано вручну; перевірено відсутність data race за допомогою ThreadSanitizer.